Case Study on the Analysis of SDLC Models Applied to a Real-World Software Application

AIM

To analyze the Software Development Life Cycle (SDLC) models used in the development of a real-world software application and identify the most suitable SDLC model.

OBJECTIVE

- Study different SDLC models.
- Analyze the development process of a real-world software application.
- Compare the advantages and limitations of various SDLC models.
- Recommend the most appropriate SDLC model for the application.

SELECTED SOFTWARE APPLICATION

Supply Chain & Logistics: Automated Courier Delivery Fee Matrix (Low-Code / Deluge Driven)

OVERVIEW

An Automated Courier Delivery Fee Matrix system allows operational desks and logistics providers to:

- Input parcel attributes (dimensions, physical weight).
- Compute custom volumetric metrics dynamically using mathematical scripts.
- Format and clean dirty tracking strings or alphanumeric postal patterns.
- Direct cross-table data lookups to isolate localized shipping fee structures.
- Disseminate automatic notifications and populate ledger payloads.

FUNCTIONAL REQUIREMENTS

Order & Shipment Ingestion Module

- Capture destination postal fields and baseline shipping classifications.
- Input package length, width, and height vectors.
- Enforce input validation locks preventing zero-value structural bounds.

Deluge Scripting Core Mechanics Module

- Dynamic execution loops parsing multiple parcel arrays.
- Text alignment subroutines converting string variables to standardized formats.
- Real-time volumetric weights extraction via mathematical algorithms.

Logistics Zone Lookup Module

- Relational pointer hooks referencing primary geographic fee registers.
- Dynamic cross-table lookups isolating cost overrides per zone tier.

Financial Billing & Alerts Module

- Final matrix cost aggregation with automated invoice population.
- Automated transaction ledger integration and outbound dispatch notifications.

SDLC MODELS ANALYSIS

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Simple structures; easy operational management paths.	Poor handling of dynamic script changes.
Prototype	Medium	Iterative form visuals improve user layout alignment.	Increases initial low-code framework outlays.
Spiral	High	Deep risk isolation vectors for core API calculations.	Complex engineering architectures for basic platforms.
Agile	Very High	Continuous iterative adjustments based on carrier matrix shifts.	Demands rigorous user testing and clean script logic.
DevOps	Very High	Automated integration triggers ensure continuous functional builds.	Demands mature script lifecycle pipeline controls.

WHY AGILE IS BEST?

Reasons

- Logistic carrier fee parameters and dynamic pricing rules scale continuously.
- New custom functions are integrated without causing system disruptions.
- Algorithmic updates are released efficiently to counter zone changes.
- Continuous user performance evaluations fine-tune data parsing flows.

AGILE DEVELOPMENT PROCESS

Requirement Gathering → Sprint Planning → Script Design → Deluge Development → Testing Blocks → Automated Deployment → Operations Feedback → Next Iteration Sprint Loop.

SDLC PHASES

1. Requirement Analysis

Isolate geometric equations, validation conditions, and routing logic matrices. Deliverable: Detailed low-code programmatic scope specification.

2. System Design

Define field normalization indices, form workflows, database schemas, and relational links. Deliverable: Platform schema blueprints.

3. Development

Write custom functions using low-code/Deluge structures on Zoho platforms tied to PostgreSQL lookup tables. Deliverables: Operational logic layers.

4. Testing

Perform rigorous validation routines, verification algorithms, and dynamic value testing.

Test Case	Expected Result
Normal Form Ingestion Check	Data formats properly; parameters process successfully.
Zero-Value Ingestion Block	Platform flags fields; execution path halts with error alerts.
Zone Multi-Table Lookup Query	System captures zone keys and injects base pricing fields.
Volumetric Formula Matrix	Custom calculation matches baseline expected values accurately.

5. Deployment

Launch cloud application modules, activate live mobile entry dashboards, and initialize webhook triggers.

6. Maintenance

Execute regular validation updates, logical enhancements, formula tuning, and custom script monitoring parameters.

COMPARISON OF SDLC MODELS

Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Req. Adaptability	Poor	Good	Excellent	Excellent	Excellent
Feedback Cycles	Low	High	High	Very High	Very High
Risk Isolation	Low	Medium	High	Medium	High
Deployment Velocity	Slow	Medium	Medium	Fast	Very Fast
Capital Outlay	Low	Medium	High	Medium	Medium
App Suitability	No	Partially	Yes	Best	Best

RESULTS

The platform requires real-time data calculations and formula flexibility. Agile SDLC supports continuous loop deployments, rapid visual forms building, and fluid programming updates. DevOps methodologies maximize automated syntax testing and version stability. Combining Agile with continuous automated operations represents the most suitable choice.

CONCLUSION

Selecting an appropriate lifecycle framework strictly relies on individual platform architecture goals. For low-code environments running critical calculation triggers, Agile provides high adaptability, quick rollout iterations, and structural transparency, while DevOps optimizes runtime testing and deployment safety metrics. This architecture minimizes data faults, improves velocity, and secures operational workflows.